Technical Diary

1. Project Aim

The goal of this project was to create **synthetic genomic datasets** at three levels:

1. **Raw data** – FASTQ
2. **Processed data** – BAM
3. **Summarised data** – VCF

using publicly available **Genome in a Bottle (GIAB)** sequencing data.
These datasets are intended for **demo, training, and long-term archival simulation**.

Environment used in this

- Jupyter Notebook
- WSL bash

In the stage of splitting FASTQ files, we used jupyter notebook with python.

2. Data Source

Genome in a Bottle (GIAB)
Sample used: **HG005**

Download here - https://zenodo.org/records/7310196

Data type:

- Whole Genome Sequencing (WGS)
- Paired-end reads (R1 & R2)

FASTQ files were downloaded from GIAB and stored locally.

Paired-end reads:

- `HG005_Son.R1.fastq.gz`
- `HG005_Son.R2.fastq.gz`

Verifying FASTQ format in jupyter notebook

```
r1_path = "HG005_Son.R1.10X.fastq"

r2_path = "HG005_Son.R2.10X.fastq"

def peek_first_read(file_path, label):

    print(f"--- First read of {label} ---")

    with open(file_path) as f:

        for _ in range(4):

            print(f.readline().strip())

    print("\n")

peek_first_read(r1_path, "R1")

peek_first_read(r2_path, "R2")
```

getting the result output as

R1

```
@D00360:77:H2YHWBCXX:1:1101:1052:2117 1:N:0:5

TAACTGTTTGTTTGAGTGTTACAGTGCTTTTGCCTATGGGAACGTCAGTGTTACTCAAGAAAAAGTCACATATGGAGGG
TGGAGGCTGCTAAAATCATGTCCTGAAGGCTGACCCAGTTCGGTGTGTTGAGGATTAGAGTCTTTCTGCTGACATCAAC
AAGTATGATCTGAGCATTTATAAGGAAATGCTCTTATATGGGGCAAAATGGAACCGTGATGCTTGGGGTAAAGTTTATC
AGCATCACCATAT

+

DDDDDIIIIGHIIIIIFHIIIIIIIIHIIIIIIIIIIIGGHGHIHIIIIHIIIIIIII?EHEHHHHHIHIIIIIG@HHI
HHHIIIHIIIIIHIIIIIIHIIHHIIIIIIIIIIIHIIIIIIIIIHIIIIIGHHHFHHIIIIIIIIIIHCGHGHIIIHEHI
IGIHGHHFGHI?E?F?EHHHHHHIIIIGIGEHHCHGHEHGE/E/CEGHHHHIHIFHIIIIFGIEH=?HH@8BG.GHCGE
H?GEHCHEHHI?8
```

R2

```
@D00360:77:H2YHWBCXX:1:1101:1052:2117 2:N:0:5
AGTTGTTCCCCATGGCAGACACTCTTATCTCCTTTGAGTTAGGGAACATGGGATTTCTTTCCTTCTTAACATCCCTGCA
TCCAGGTAGATATGGATTAGGGCCCAAGGTTGTTCTGATTTCTTGTTTATCCTCTTAGAAAATCACTGACAGAAAAAAA
```

```
GTATTACTAACTCAAAAAGCCTCAGCCAAATTCCCAGCTCTAATATGTACTAAATATTTCTTCAGCCACTTATCAGCTT
GATAATTTTTTTC
+
DDDB@HHIIIIIIIIIGEHHHIHIHIIIIIIIIIIIIHIIIIIIIHIIIIIGEECHHIIIIIIHIGHHIHGHHHHHIIHHI
IHHIHHIHHHHHIIIHEEHHICDGHEHHEHIIIHHIHGHHHHHIGHHH@HHGIIFHHCGHEEGF?FFHHHCFGEEFFHI
IIHGGFHHDC0<CFCHHHHHEHHGCHIC@GHHECHH?@/@CG<D/<<7<GFHCGHFCGHHEGHFEHHFEAH.BFH@.FE
CEHCEAGF9FCD-
```

3. Creating Synthetic Samples by Splitting

Splitting into 5 samples paired reads as 10 files

In jupyter cell:

```
r1_path = "HG005_Son.R1.10X.fastq"
r2_path = "HG005_Son.R2.10X.fastq"


os.makedirs("results", exist_ok=True)

def fastq_generator(file_path):
    """Yield one read (4 lines) at a time"""
    with open(file_path) as f:
        while True:
            record = [f.readline().strip() for _ in range(4)]
            if not record[0]:
                break
            yield record

r1_gen = fastq_generator(r1_path)
r2_gen = fastq_generator(r2_path)

#samples and reads per sample
samples = ["sample1", "sample2", "sample3", "sample4", "sample5"]
reads_per_sample = 1000000

# Splitting
for sample in samples:
    r1_out = f"results/{sample}_R1.fastq"
```

```
    r2_out = f"results/{sample}_R2.fastq"

    with open(r1_out, "w") as o1, open(r2_out, "w") as o2:
        for _ in range(reads_per_sample):
            r1_record = next(r1_gen)
            r2_record = next(r2_gen)
            o1.write("\n".join(r1_record) + "\n")
            o2.write("\n".join(r2_record) + "\n")

    print(f"{sample} created: {r1_out} + {r2_out}")

print("FASTQ splitting complete!")

#Results
files = os.listdir("results")
print("Files in results folder:", files)

with open("results/sample1_R1.fastq") as f:
    print("\nFirst read in sample1_R1.fastq:")
    for _ in range(4):
        print(f.readline().strip()) #this is optional
```

Getting the result as

sample1_R1.fastq
sample1_R2.fastq
sample2_R1.fastq
sample2_R2.fastq
sample3_R1.fastq
sample3_R2.fastq
sample4_R1.fastq
sample4_R2.fastq
sample5_R1.fastq
sample5_R2.fastq

4. Read Count Validation

Paired-end integrity confirmed if the read count is 100k for each sample.

```
def count_reads(file):
    return sum(1 for _ in open(file)) // 4
```

```
for f in os.listdir("results"):
    print(f"{f}: {count_reads(os.path.join('results', f))} reads")
```

If the reads count for each individual file is 100k and the same, this is correct. It should have the same read counts for each paired-ends sample.

FASTQ files are available here -

https://wehieduau.sharepoint.com/:f:/r/sites/StudentInternGroupatWEHI/Shared%20Documents/Data%20Commons/2026%20Summer/Data%20Workflow/FASTQ%20Splitting?csf=1&web=1&e=4pM4BG

5. Processed Data (BAM)

Tools Used: BWA, SamTools
System: Windows Subsystem for Linux (WSL) -> Ubuntu

Aim: To convert the FASTQ files of 5 samples into BAM files.

🔹 Stage 1 (initial plan)

What we *intended* to use:

Full human reference genome – GRCh38 (hg38)
(file like `GRCh38.fasta` / `GCF_000001405.40_GRCh38…`)

Why:

- This is the standard reference for human DNA
- Correct for real biological analysis

What happened:

- Indexing required high RAM (32-64 GB) but 8GB was present.
- `bwa index` was killed
- `.sa` file was never created
- Alignment failed (0 KB SAM)

➡️ **So this reference was planned, but never successfully used**

🔹 Stage 2 (attempt to fix it)

What we tried next:

Pre-built GRCh38 BWA indexes (Illumina / Ensembl / UCSC)

Why:

- To avoid local indexing
- To still use full human genome

What happened:

- Network blocks (SSL, rsync, FTP paths)
- Files not accessible from network

➡️ **Still no usable full-genome reference**

🔹 Stage 3 (what ACTUALLY worked successfully)

Human chromosome 22 from GRCh38 (hg38)
File: `chr22.fasta`

Why this was chosen:

- Small enough to index on laptop (~50 MB)
- Same biology, same pipeline
- Works for learning, demos, exams

This is the reference used for:

- `bwa index -> indexing the reference genome for faster alignment`
- `bwa mem -> alignment with sample FASTQ files`
- `sample.sam -> output of the alignment`
- `Sample.bam -> processed files`
- `Sample_sorted.bam -> orders reads by chromosome and genomic position. Needed for visualization.`

➡️ **This is the first reference that actually worked end-to-end**

Limitations:

In this analysis, **only chromosome 22** of the human GRCh38 (hg38) reference genome was used instead of the complete genome. **As a result, sequencing reads originating from other chromosomes could not be aligned and were marked as unmapped**, leading to a lower overall mapping percentage. The analysis was therefore restricted to genomic regions present on chromosome 22, and genome-wide interpretations such as whole-genome variant detection, coverage analysis, and cross-chromosomal comparisons were not possible. While this approach is **not suitable for comprehensive biological or clinical conclusions**, it is **sufficient for demonstrating** and understanding the FASTQ-to-BAM alignment workflow and associated bioinformatics processing steps.

Workflow:

dwaipayan123@LAPTOP-HVNDPIBE:/mnt/c/Users/Dwaipayan/Downloads/FASTQ Splitting/FASTQ Splitting$

> cd /mnt/c/Users/Dwaipayan/Downloads/FASTQ\ Splitting/FASTQ\ Splitting **#change pwd**

> wget http://hgdownload.soe.ucsc.edu/goldenPath/hg38/chromosomes/chr22.fa.gz **#download the chromosome 22 of the reference genome.**

--2026-01-11 12:50:35-- http://hgdownload.soe.ucsc.edu/goldenPath/hg38/chromosomes/chr22.fa.gz

Resolving hgdownload.soe.ucsc.edu (hgdownload.soe.ucsc.edu)... 128.114.119.163

Connecting to hgdownload.soe.ucsc.edu (hgdownload.soe.ucsc.edu)|128.114.119.163|:80... connected.

HTTP request sent, awaiting response... 200 OK

Length: 12255678 (12M) [application/x-gzip]

Saving to: ‘chr22.fa.gz’

chr22.fa.gz
100%[===>] 11.69M 140KB/s in 2m 42s

2026-01-11 12:53:06 (73.7 KB/s) - ‘chr22.fa.gz’ saved [12255678/12255678]

> gunzip chr22.fa.gz **# unzip the folder containing the reference genome.**

> mv chr22.fa chr22.fasta **#change file format to .fasta**

> ls -lh chr22.fasta **# checking size**

-rwxrwxrwx 1 dwaipayan123 dwaipayan123 50M Jan 24 2014 chr22.fasta

> bwa index chr22.fasta **#indexing the reference genome.**

[bwa_index] Pack FASTA... 1.13 sec

[bwa_index] Construct BWT for the packed sequence...

[BWTIncCreate] textLength=101636936, availableWord=19151484

[BWTIncConstructFromPacked] 10 iterations done. 31590664 characters processed.

[BWTIncConstructFromPacked] 20 iterations done. 58359704 characters processed.

[BWTIncConstructFromPacked] 30 iterations done. 82148056 characters processed.

[BWTIncConstructFromPacked] 40 iterations done. 101636936 characters processed.

[bwt_gen] Finished constructing BWT in 40 iterations.

[bwa_index] 48.31 seconds elapse.

[bwa_index] Update BWT... 0.23 sec

[bwa_index] Pack forward-only FASTA... 0.35 sec

[bwa_index] Construct SA from BWT and Occ... 17.29 sec

[main] Version: 0.7.17-r1188

[main] CMD: bwa index chr22.fasta

[main] Real time: 72.057 sec; CPU: 67.329 sec

> bwa mem chr22.fasta sample1_R1.fastq sample1_R2.fastq > sample1.sam

#alignment command giving reference genome and the paired-ended FASTQ files in the input.

[M::bwa_idx_load_from_disk] read 0 ALT contigs

[M::process] read 2000 sequences (496901 bp)...

[M::mem_pestat] # candidate unique pairs for (FF, FR, RF, RR): (0, 19, 1, 0)

[M::mem_pestat] skip orientation FF as there are not enough pairs

[M::mem_pestat] analyzing insert size distribution for orientation FR...

[M::mem_pestat] (25, 50, 75) percentile: (193, 439, 561)

[M::mem_pestat] low and high boundaries for computing mean and std.dev: (1, 1297)

[M::mem_pestat] mean and std.dev: (398.00, 285.45)

[M::mem_pestat] low and high boundaries for proper pairs: (1, 1665)

[M::mem_pestat] skip orientation RF as there are not enough pairs

[M::mem_pestat] skip orientation RR as there are not enough pairs

[M::mem_process_seqs] Processed 2000 reads in 2.685 CPU sec, 2.908 real sec

[main] Version: 0.7.17-r1188

[main] CMD: bwa mem chr22.fasta sample1_R1.fastq sample1_R2.fastq

[main] Real time: 4.010 sec; CPU: 2.939 sec

> ls -lh sample1.sam **#checking file size**

-rwxrwxrwx 1 dwaipayan123 dwaipayan123 1.3M Jan 11 12:58 sample1.sam

> samtools view -Sb sample1.sam > sample1.bam **#convert sam to bam file**

> samtools sort sample1.bam -o sample1_sorted.bam **#converting bam to sorted_bam**

> samtools index sample1_sorted.bam **#converting bam to .bai**

> bwa mem chr22.fasta sample2_R1.fastq sample2_R2.fastq > sample2.sam **#repeat for sample 2**

[M::bwa_idx_load_from_disk] read 0 ALT contigs

[M::process] read 2000 sequences (497580 bp)...

[M::mem_pestat] # candidate unique pairs for (FF, FR, RF, RR): (0, 25, 1, 0)

[M::mem_pestat] skip orientation FF as there are not enough pairs

[M::mem_pestat] analyzing insert size distribution for orientation FR...

[M::mem_pestat] (25, 50, 75) percentile: (132, 324, 519)

[M::mem_pestat] low and high boundaries for computing mean and std.dev: (1, 1293)

[M::mem_pestat] mean and std.dev: (357.64, 250.16)

[M::mem_pestat] low and high boundaries for proper pairs: (1, 1680)

[M::mem_pestat] skip orientation RF as there are not enough pairs

[M::mem_pestat] skip orientation RR as there are not enough pairs

[M::mem_process_seqs] Processed 2000 reads in 2.869 CPU sec, 3.110 real sec

[main] Version: 0.7.17-r1188

[main] CMD: bwa mem chr22.fasta sample2_R1.fastq sample2_R2.fastq

[main] Real time: 4.661 sec; CPU: 3.237 sec

> bwa mem chr22.fasta sample3_R1.fastq sample3_R2.fastq > sample3.sam **#repeat for sample 3**

[M::bwa_idx_load_from_disk] read 0 ALT contigs

[M::process] read 2000 sequences (497875 bp)...

[M::mem_pestat] # candidate unique pairs for (FF, FR, RF, RR): (0, 26, 1, 0)

[M::mem_pestat] skip orientation FF as there are not enough pairs

[M::mem_pestat] analyzing insert size distribution for orientation FR...

[M::mem_pestat] (25, 50, 75) percentile: (456, 648, 807)

[M::mem_pestat] low and high boundaries for computing mean and std.dev: (1, 1509)

[M::mem_pestat] mean and std.dev: (635.08, 253.10)

[M::mem_pestat] low and high boundaries for proper pairs: (1, 1860)

[M::mem_pestat] skip orientation RF as there are not enough pairs

[M::mem_pestat] skip orientation RR as there are not enough pairs

[M::mem_process_seqs] Processed 2000 reads in 2.980 CPU sec, 3.232 real sec

[main] Version: 0.7.17-r1188

[main] CMD: bwa mem chr22.fasta sample3_R1.fastq sample3_R2.fastq

[main] Real time: 4.912 sec; CPU: 3.372 sec

> bwa mem chr22.fasta sample4_R1.fastq sample4_R2.fastq > sample4.sam **#repeat for sample 4**

[M::bwa_idx_load_from_disk] read 0 ALT contigs

[M::process] read 2000 sequences (497502 bp)...

[M::mem_pestat] # candidate unique pairs for (FF, FR, RF, RR): (0, 25, 2, 0)

[M::mem_pestat] skip orientation FF as there are not enough pairs

[M::mem_pestat] analyzing insert size distribution for orientation FR...

[M::mem_pestat] (25, 50, 75) percentile: (357, 484, 623)

[M::mem_pestat] low and high boundaries for computing mean and std.dev: (1, 1155)

[M::mem_pestat] mean and std.dev: (454.38, 199.47)

[M::mem_pestat] low and high boundaries for proper pairs: (1, 1421)

[M::mem_pestat] skip orientation RF as there are not enough pairs

[M::mem_pestat] skip orientation RR as there are not enough pairs

[M::mem_process_seqs] Processed 2000 reads in 2.518 CPU sec, 2.732 real sec

[main] Version: 0.7.17-r1188

[main] CMD: bwa mem chr22.fasta sample4_R1.fastq sample4_R2.fastq

[main] Real time: 3.846 sec; CPU: 2.815 sec

> bwa mem chr22.fasta sample5_R1.fastq sample5_R2.fastq > sample5.sam **#repeat for sample 5**

[M::bwa_idx_load_from_disk] read 0 ALT contigs

[M::process] read 2000 sequences (497579 bp)...

[M::mem_pestat] # candidate unique pairs for (FF, FR, RF, RR): (0, 20, 0, 0)

[M::mem_pestat] skip orientation FF as there are not enough pairs

[M::mem_pestat] analyzing insert size distribution for orientation FR...

[M::mem_pestat] (25, 50, 75) percentile: (270, 568, 755)

[M::mem_pestat] low and high boundaries for computing mean and std.dev: (1, 1725)

[M::mem_pestat] mean and std.dev: (499.25, 263.88)

[M::mem_pestat] low and high boundaries for proper pairs: (1, 2210)

[M::mem_pestat] skip orientation RF as there are not enough pairs

[M::mem_pestat] skip orientation RR as there are not enough pairs

[M::mem_process_seqs] Processed 2000 reads in 3.085 CPU sec, 3.348 real sec

[main] Version: 0.7.17-r1188

[main] CMD: bwa mem chr22.fasta sample5_R1.fastq sample5_R2.fastq

[main] Real time: 4.455 sec; CPU: 3.346 sec

> **# sorted_bam for sample 2-5**

samtools sort sample2.bam -o sample2_sorted.bam

samtools sort sample3.bam -o sample3_sorted.bam

samtools sort sample4.bam -o sample4_sorted.bam

samtools sort sample5.bam -o sample5_sorted.bam

> **# bai files for sample 2-5**

samtools index sample2_sorted.bam

samtools index sample3_sorted.bam

samtools index sample4_sorted.bam

samtools index sample5_sorted.bam

SETBACK !!!

We have produced fastq files with reads that might not be enough, so we have reproduced fastq files with 100K and 10M reads:

Converting those files to BAM files:

dwaipayan123@LAPTOP-HVNDPIBE:~$ cd /mnt/c/Users/Dwaipayan/Downloads/FASTQ\ Splitting/

dwaipayan123@LAPTOP-HVNDPIBE:/mnt/c/Users/Dwaipayan/Downloads/FASTQ Splitting$ mkdir FASTQ_Splitting

dwaipayan123@LAPTOP-HVNDPIBE:/mnt/c/Users/Dwaipayan/Downloads/FASTQ Splitting$ cd FASTQ_Splitting

```
dwaipayan123@LAPTOP-HVNDPIBE:/mnt/c/Users/Dwaipayan/Downloads/FASTQ
Splitting/FASTQ_Splitting$ wget
http://hgdownload.soe.ucsc.edu/goldenPath/hg38/chromosomes/chr22.fa.gz

--2026-01-15 13:58:51--
http://hgdownload.soe.ucsc.edu/goldenPath/hg38/chromosomes/chr22.fa.gz

Resolving hgdownload.soe.ucsc.edu (hgdownload.soe.ucsc.edu)... 128.114.119.163

Connecting to hgdownload.soe.ucsc.edu (hgdownload.soe.ucsc.edu)|128.114.119.163|:80...
connected.

HTTP request sent, awaiting response... 200 OK

Length: 12255678 (12M) [application/x-gzip]

Saving to: ‘chr22.fa.gz’



chr22.fa.gz                100%[===================================================>]
11.69M  1.36MB/s    in 9.4s



2026-01-15 13:58:59 (1.24 MB/s) - ‘chr22.fa.gz’ saved [12255678/12255678]



dwaipayan123@LAPTOP-HVNDPIBE:/mnt/c/Users/Dwaipayan/Downloads/FASTQ
Splitting/FASTQ_Splitting$ gunzip chr22.fa.gz

dwaipayan123@LAPTOP-HVNDPIBE:/mnt/c/Users/Dwaipayan/Downloads/FASTQ
Splitting/FASTQ_Splitting$ mv chr22.fa chr22.fasta

dwaipayan123@LAPTOP-HVNDPIBE:/mnt/c/Users/Dwaipayan/Downloads/FASTQ
Splitting/FASTQ_Splitting$  ls -lh chr22.fasta

-rwxrwxrwx 1 dwaipayan123 dwaipayan123 50M Jan 24  2014 chr22.fasta

dwaipayan123@LAPTOP-HVNDPIBE:/mnt/c/Users/Dwaipayan/Downloads/FASTQ
Splitting/FASTQ_Splitting$ bwa index chr22.fasta

[bwa_index] Pack FASTA... 0.50 sec

[bwa_index] Construct BWT for the packed sequence...

[BWTIncCreate] textLength=101636936, availableWord=19151484

[BWTIncConstructFromPacked] 10 iterations done. 31590664 characters processed.

[BWTIncConstructFromPacked] 20 iterations done. 58359704 characters processed.
```

[BWTIncConstructFromPacked] 30 iterations done. 82148056 characters processed.

[BWTIncConstructFromPacked] 40 iterations done. 101636936 characters processed.

[bwt_gen] Finished constructing BWT in 40 iterations.

[bwa_index] 47.95 seconds elapse.

[bwa_index] Update BWT... 0.22 sec

[bwa_index] Pack forward-only FASTA... 0.31 sec

[bwa_index] Construct SA from BWT and Occ... 17.42 sec

[main] Version: 0.7.17-r1188

[main] CMD: bwa index chr22.fasta

[main] Real time: 67.633 sec; CPU: 66.397 sec

dwaipayan123@LAPTOP-HVNDPIBE:/mnt/c/Users/Dwaipayan/Downloads/FASTQ Splitting/FASTQ_Splitting$ bwa mem chr22.fasta sample1_R1.fastq sample1_R2.fastq > sample1.sam

[M::bwa_idx_load_from_disk] read 0 ALT contigs

[M::process] read 40214 sequences (10000331 bp)...

[M::process] read 40208 sequences (10000129 bp)...

[M::mem_pestat] # candidate unique pairs for (FF, FR, RF, RR): (1, 443, 13, 0)

[M::mem_pestat] skip orientation FF as there are not enough pairs

[M::mem_pestat] analyzing insert size distribution for orientation FR...

[M::mem_pestat] (25, 50, 75) percentile: (316, 529, 656)

[M::mem_pestat] low and high boundaries for computing mean and std.dev: (1, 1336)

[M::mem_pestat] mean and std.dev: (489.86, 251.04)

[M::mem_pestat] low and high boundaries for proper pairs: (1, 1676)

[M::mem_pestat] analyzing insert size distribution for orientation RF...

[M::mem_pestat] (25, 50, 75) percentile: (232, 245, 279)

[M::mem_pestat] low and high boundaries for computing mean and std.dev: (138, 373)

[M::mem_pestat] mean and std.dev: (251.90, 37.19)

```
[M::mem_pestat] low and high boundaries for proper pairs: (91, 420)
[M::mem_pestat] skip orientation RR as there are not enough pairs
[M::mem_pestat] skip orientation RF
[M::mem_process_seqs] Processed 40214 reads in 61.922 CPU sec, 61.830 real sec
[M::process] read 40222 sequences (10000266 bp)...
[M::mem_pestat] # candidate unique pairs for (FF, FR, RF, RR): (1, 438, 24, 2)
[M::mem_pestat] skip orientation FF as there are not enough pairs
[M::mem_pestat] analyzing insert size distribution for orientation FR...
[M::mem_pestat] (25, 50, 75) percentile: (284, 560, 700)
[M::mem_pestat] low and high boundaries for computing mean and std.dev: (1, 1532)
[M::mem_pestat] mean and std.dev: (516.23, 273.58)
[M::mem_pestat] low and high boundaries for proper pairs: (1, 1948)
[M::mem_pestat] analyzing insert size distribution for orientation RF...
[M::mem_pestat] (25, 50, 75) percentile: (223, 281, 460)
[M::mem_pestat] low and high boundaries for computing mean and std.dev: (1, 934)
[M::mem_pestat] mean and std.dev: (286.23, 116.00)
[M::mem_pestat] low and high boundaries for proper pairs: (1, 1171)
[M::mem_pestat] skip orientation RR as there are not enough pairs
[M::mem_process_seqs] Processed 40208 reads in 73.484 CPU sec, 73.177 real sec
…
…
…
…
[main] Version: 0.7.17-r1188
[main] CMD: bwa mem chr22.fasta sample1_R1.fastq sample1_R2.fastq
[main] Real time: 3229.141 sec; CPU: 3241.031 sec
```

dwaipayan123@LAPTOP-HVNDPIBE:/mnt/c/Users/Dwaipayan/Downloads/FASTQ Splitting/FASTQ_Splitting$ ls -lh sample1.sam

-rwxrwxrwx 1 dwaipayan123 dwaipayan123 1.3G Jan 15 14:57 sample1.sam

dwaipayan123@LAPTOP-HVNDPIBE:/mnt/c/Users/Dwaipayan/Downloads/FASTQ Splitting/FASTQ_Splitting$ samtools view -Sb sample1.sam > sample1.bam

dwaipayan123@LAPTOP-HVNDPIBE:/mnt/c/Users/Dwaipayan/Downloads/FASTQ Splitting/FASTQ_Splitting$ samtools sort sample1.bam -o sample1_sorted.bam

[bam_sort_core] merging from 1 files and 1 in-memory blocks...

dwaipayan123@LAPTOP-HVNDPIBE:/mnt/c/Users/Dwaipayan/Downloads/FASTQ Splitting/FASTQ_Splitting$ samtools index sample1_sorted.bam

Variant Calling (VCF)

Tools we will use

- `bcftools`

We will use **bcftools**, which is simple and standard.

After getting BAM, sorted BAM and BAI, proceed to VCF. In your data directory, make sure sorted BAM and BAI are ready and stable for each sample as something like this

sample1.sorted.bam

sample1.sorted.bam.bai

Now set reference genome path

Bash

REF=/mnt/d/bioproject/reference/chr22.fa

Generate mpileup (intermediate BCF)

For each sample

Bash - bcftools mpileup -f $REF sample1_sorted.bam -Ou -o sample1.mpileup.bcf bcftools mpileup -f $REF sample2_sorted.bam -Ou -o sample2.mpileup.bcf

bcftools mpileup -f $REF sample3_sorted.bam -Ou -o sample3.mpileup.bcf

bcftools mpileup -f $REF sample4_sorted.bam -Ou -o sample4.mpileup.bcf

bcftools mpileup -f $REF sample5_sorted.bam -Ou -o sample5.mpileup.bcf

Variant calling (VCF generation)

Using bcftools call with multiallelic calling enabled.

For each sample

Bash - bcftools call -mv -Ov -o sample1.vcf sample1.mpileup.bcf

Same process for each sample 2 to 5.

After that, verify VCF files exist and are non-zero with ls -lh *.vcf

If file sizes are ~5MB, they are non-zero.

Inspect VCF headers

Bash - head sample1.vcf

If observes

##fileformat=VCFv4.2

##bcftoolsVersion=1.19

##reference=file:///mnt/d/bioproject/reference/chr22.fa

##contig=<ID=chr22,length=50818468>

This confirms VCF format and reference genome.

Inspect variant entries with no header

Bash

```
grep -v "^#" sample1.vcf | head
```

Example variant line

chr22 10514057 . A G 10.7923 . DP=1;AC=2;AN=2;MQ=60 GT:PL 1/1:40,3,0

chr22 10514602 . C T 10.7923 . DP=1;AC=2;AN=2;MQ=60 GT:PL 1/1:40,3,0

If this is confirmed, it means VCF has actual variants and genotype information is relevant.